Інструкція до практичної роботи №73

ТЕМА. "АДРЕСИ ДАНИХ. ВКАЗІВНИКИ. ДИНАМІЧНА ПАМ'ЯТЬ"

МЕТА: Ознайомитись з адресами даних, вказівниками, динамічною пам'яттю і файлами. Набути практичних навичок створення програм з введенням даних з файлів і виведенням даних у файл.

Теоретичні відомості

Адреси даних.

Для розв'язування специфічних для мови С++ задач (робота з масивами, побудова графічних зображень і динамічних ефектів) потрібно знати не тільки значення деякої змінної, але й її адресу в оперативній пам'яті. Для визначення адреси даного у пам'яті є операція визначення адреси

&<назва даного>

Приклад 1. Розглянемо фрагмент програми

```
int a = 25;
cout« "Значення змінної a = " « a « "\n";
cout « "Адресою змінної a є " « &a;
```

У результаті виконання цих команд на екрані одержимо:

Значення змінної a = 25

Адресою змінної a є 0x*af72254*

Адреси зображуються шістнадцятковими числами і під час кожного виконання програми можуть бути різними.

Вказівники.

У мові С++ є ще один засіб визначення адреси даного — це вказівники. ***Вказівник*** *— це змінна, значенням якої є адреса*. Вона вказує на початок області оперативної пам'яті, де зберігається дане. Вказівники дають змогу оперувати не з іменами даних, а з їх адресами. Вказівники утворюють так:

<тип даного> *<назва вказівника»;

Можна створювати вказівники на сталі, змінні, функції, інші вказівники тощо. Особливо ефективною є робота з вказівниками на рядки та масиви.

Наприклад, int *nomer;

Float *rist, *prtA, *prtB;

Тут оголошено вказівник на цілий тип nomer і вказівники на дійсний тип rist, prtA, prtB. Надати значення вказівникам можна так:

<назва вказівника> = <адреса змінної»;

Приклад 2. Нехай у програмі оголошені змінні

```
int n = 10; float stud1, stud2, stud3;
```

Тоді можна оголосити вказівники nomer і rist на ці змінні:

```
nomer = &n; rist = &stud1; rist = &stud3;
```

Отримати вмістиме комірки, на яку вказує вказівник, можна за допомогою конструкції *<вказівник>, наприклад *rist є значенням змінної stud3. Вказівники застосовують також для надання значень змінним:

```
rist = &stud1;
    rist = 1.65;       //У комірку пам'яті для змінної stud1 буде занесено число 1.65
```

Тут вказівник rist вказуватиме на адресу змінної stud1, а власне змінній stud1 буде присвоєно значення 1.65.

Приклад 3. Обчислити довжини (в байтах) вказівників на

різні типи даних можна так:

```
int *prt_i; double *prt_d;
cout << "\nНа цілий тип “ << sizeof(prt_i);
cout << "\nНа дійсний тип “ << sizeof(prt_d);
```

Переконайтесь, що усі вище описані вказівники у пам’яті комп’ютера мають однаковий обсяг.

Вказівники можна переадресовувати. Зокрема, у прикладі 2 вказівнику rist спочатку присвоєно адресу змінної stud1, а потім - змінної stud3. Це правило не діє, якщо вказівник є сталою. Сталий вказівник, що вказуватимемо на одну і ту ж адресу, оголошують так: const int *rik;

Для вказівників залежно від операційної системи та версії компілятора резервується 2 або 4 байти в оперативній пам’яті.

Над вказівниками визначені арифметичні операції та операції порівняння, описані в табл. 1.

Таблиця 1. Операції, визначені над вказівниками

Операція	Приклади і пояснення
= =,!=, >=, <=, >, <	Порівнює значення двох вказівників (адреси, на які вони вказують). Наприклад, якщо вказівники вказують на одне і те ж саме дане, то результатом порівняння Vk1 == Vk2 буде істина, інакше - хибність
-	Vk1–Vk2. Використовується для визначення кількості елементів, які наявні між двома вказівниками
+, -	Vk1 + k, Vk1 - k. Знаходить вказівник, який зміщений відносно даного на k одиниць

Динамічна пам’ять. Команди new і delete.

Значення даних у пам’яті комп’ютера можна зберігати в області даних (статична пам’ять), у стеку або в динамічній пам’яті ("на купі"). Усі змінні, які ми розглядали досі, - статичні. Під час їх оголошення система автоматично надає для зберігання їхніх значень певний обсяг оперативної пам’яті, і цей розподіл пам’яті залишається незмінним протягом виконання програми. Пам’ять, надана цим змінним, резервується (фіксується) у середині exe-файлу відкомпільованої програми. Навіть якщо не всі змінні будуть використані у програмі, пам’ять буде зарезервована, тобто використана неефективно. Пам’ять, надана таким змінним, вивільняється лише після виконання програми. Тому в оперативній пам’яті можна розмістити лише обмежену кількість даних.

Однак є задачі, де заздалегідь невідомо, скільки змінних потрібно для їхнього розв’язування, а, отже, який обсяг пам’яті потрібно зарезервувати, або задачі, у яких,

навпаки, заздалегідь відомо, що змінних буде багато, наприклад, задачі опрацювання масивів великих розмірів. У таких випадках застосовують динамічну організацію пам'яті.

Принцип динамічної організації пам'яті полягає у тому, що для змінних надається пам'ять за необхідністю (за вказівкою програміста). Далі ці змінні опрацьовують і в потрібний момент пам'ять вивільняють (знову за вказівкою програміста). Такі змінні називаються динамічними.

Для роботи з динамічними змінними використовують вказівники. Для виділення динамічної пам'яті застосовується команда new так: **<тип вказівника> *<назва> = new <тип змінної>;**

Дія команди new. Для відповідного типу змінної автоматично надається необхідна неперервна ділянка пам'яті. Команда new повертає обсяг цієї ділянки, а вказівник вказує на її початок. Наприклад, щоб зарезервувати у пам'яті комп'ютера область для зберігання значення цілого типу, застосовують таку команду: int *prt = new int;.

Надати ділянку пам'яті й відразу занести у неї значення можна так:

int *prt2 = new float(3.14);.

У адресу, на яку показує prt2, буде занесено число 3,14.

З динамічною змінною можна виконувати операції, визначені для даних відповідного базового типу.

Після опрацювання динамічних змінних пам'ять необхідно вивільнити, а відповідний вказівник занулити. Якщо цього не зробити, то пам'ять можна вичерпати. Вивільняють пам'ять за допомогою команди **delete** <назва вказівника**>;**

Щоб вказівник не вказував на жодну ділянку пам'яті, його необхідно занулити такою командою:

<назва вказівника> = **NULL;**

Значенням (адресою) такого вказівника буде нульова адреса 0x00000000. Тут не може буте розміщене значення жодного даного.

Приклад 4. Розглянемо, як відбувається розподіл пам'яті. Надамо пам'ять для двох змінних цілого типу та присвоїмо їм деякі значення. Пізніше вивільнимо пам'ять.

```
#include <iostream.h>
void main()
{
int  *c1 = new int;        //Готуємо пам'ять для цілого числа
*c1 = 5;                   //Змінна 'c1 набуває значення 5
int *c2 = new int(7);              // Виділяється пам'ять для c2 і *c2 набуває значення 7
cout << *c1 << "\t" « *c2<< "\n";   //  Виводимо 5 та 7
                                   // Переадресація -c1 вказуватимемо на ту саму
 -c1 = c2;                 //ділянку пам'яті, що і c2
cout << *c1 << "\t << *c2 << "\n"; //  Виводимо 7 та 7
delete(c2);                //Пам'ять, надану для c2, вивільняємо
cout<<*c1 << "\n";         //Виводимо 7
}
```

Довідка 1. У стандарті ANSI мови С у модулях stdlib.h та alloc.h визначені функції malloc і calloc для надання динамічної пам'яті, а також функція free для вивільнення пам'яті. Детальніше про ці функції можна прочитати у довідниках.

Файли.

Часто виникає потреба опрацьовувати інформацію, розміщену на зовнішніх носіях (на дисках), або виводити результати програми не на екран монітора, а у файл. Прикладами таких даних є інформація про розклад руху транспорту, успішність студентів, наявність товарів у магазинах, адреси, анкетні дані тощо.

Файл - це сукупність даних, які розміщені на зовнішньому носії, зокрема, на жорсткому диску. Дані у файлі називаються елементами. Кількість даних під час опрацювання файлів не зазначають. Файли можуть містити як текстову, так і числову інформацію.

Існує декілька бібліотечних функцій, які дають змогу зчитати інформацію з файлу або спрямовувати потік виведення у файл. Функції, які розглядатимемо, описані в бібліотеці **fstream.h.** Тому для роботи з файлами насамперед необхідно підключити саме цю бібліотеку.

Зауваження 1. У модулі fstream.h також містяться описи стандартних потоків cout і cin. Тому, щоб зчитати інформацію з файлу і вивести її на екран монітора, модуль iostream.h підключати не обов'язково.

Для опрацювання файлу його необхідно відкрити, виконати потрібні дії та закрити.

Зчитування даних із файлу.

Щоб зчитати вхідні дані з файлу, необхідно оголосити файлову змінну та відкрити файл для читання так (розглянемо два способи):

ifstream <назва файлової змінної>(<зовнішня назва>, ознака1,ознака2 , ... , ознакаN);

або так:

ifstream <назва файлової змінної>;

<назва файлової змінної>**.open** (<зовнішня назва>);

Ознаки слугують для визначення прав доступу до файлу (див. пункт „Ознаки"). Ознаки зазначати необов'язково.

Відкрити зовнішній файл MyText.txt для читання з нього даних можна так:

ifstream file1("MyText.txt"); або так: **ifstream file1; file1.open("MyText.txt");**

Тут **file1** - це файлова змінна, що асоціюється у програмі з відповідним зовнішнім файлом на диску. Введення даних виконують командою введення >>. Наприклад, команда **file1** >>text;

зчитає одне дане з початку файлу MyText.txt до першого пропуску чи до символу „кінець рядка" у змінну text. Якщо треба зчитати наступне дане (яке розміщене між першим і другим пропусками), необхідно повторити попередню команду, тобто записати команди f >> text; f>> textl;

або використати команду **f >>text>>text1;.**

Отже, зчитати дані з файлу можна за допомогою команди:

<назва файлової змінної> >> <змінна 1> >> <змінна 2> >> ... >> <змінна N>

Після опрацювання файлу його потрібно закрити. Це роблять так:

<назва файлової змінної>.close()

Наприклад**, file1.close().**

Файл MyText.txt буде закрито. Після цього файлову змінну **file1** у разі потреби можна зв'язати з іншим файлом на диску.

Визначити кінець файлу можна за допомогою функції eof() (eof - end of file, кінець файлу) так:

<назва файлової змінної>.eof()

Ця функція повертає нульове значення, якщо досягнуто кінця файлу.

Приклад 5. Дано файл MyText.txt, який містить слова розділені прогалиною. Скласти програму, яка виведе на екран перші два слова.

```
#include <fstream.h>
#include <iostream.h>
#include <conio.h>
int main()
{
  char slovo1[100],slovo2[100];
 ifstream file1("MyText.txt");
  file1 >>slovo1 >> slovo2;
  cout<< slovo1<< "\n"<<slovo2;
  file1.close();
   getch();
       return 0;
}
```

Приклад 6. Дано файл MyData.txt, який містить цілі числа розділені прогалиною. Скласти програму, яка виведе на екран суму перших двох чисел з файла.

```
#include <fstream.h>
#include <iostream.h>
#include <conio.h>
int main()
{
  int a,b,s;
 ifstream file1("MyData.txt");
  file1 >>a >> b;
  s=a+b;
  cout<< a<< "+"<<b<<"="<<s;
  file1.close();
   getch();
       return 0;
}
```

Виведення даних у файл.

Відкрити файл для записування у нього даних можна так:

ofstream <назва файлової змінної>(<зовнішня назва>, ознака1 , ознака2 ,... , ознакаN>

або так:

ofstream <назва файлової змінної>;

<назва файлової змінної>**.open**(<зовнішня назва>);

Дія команд. Підключаємо потік виведення у файл і налагоджуємо зв’язок між файловою змінною у програмі та файлом на зовнішньому носії. Ознаки зазначати не

обов’язково.

Відкрити файл MyRez.txt для запису у нього даних можна так (тут file2 - файлова змінна):

ofstream file2("MyRez .txt"); або так: **ofstream file2; file2.open("MyRez .txt");**

Щоб занести дані у файл, використовують команду виведення даних <<.

Отже, записати дані у файл можна так:

<назва файлової змінної> << <змінна 1> << <змінна 2> << ... << <змінна N>;

Ознаки.

Ознаки слугують для зміни правил доступу до файлу. Ось деякі з них:

Ознака	Призначення
ios::in	Відкриває файл для читання з нього інформації. Вміст файлу зберігається
ios::out	Відкриває файл для записування інформації. Якщо файл не існує, то буде створений
ios::app	Відкриває файл для дописування інформації. Дані будуть записані у кінець файлу
ios::trunk	Якщо файл, який відкривають для записування вже існує, то його вміст буде вилучено
ios::nocreate	Забороняє створювати файл, який відкривають
ios::noreplace	Забороняє перезаписати існуючий файл

Приклад 7. Дано файл MyData.txt, який містить дійсні числа розділені прогалиною. Скласти програму, яка обчислить значення у задане формулою для перших двох чисел з даного файла (перше число – a, друге – b) . Результат виведе на екран і запише в файл MyRez.txt.

```
#include <fstream.h>
#include <iostream.h>
#include <conio.h>
#include <math.h>
int main()
{
  float a,b,y;
 ifstream file1("MyData.txt");
  file1 >>a >> b;
  y=(sqrt(a+b)+5*exp(0.2*a))/(pow(a,2)+pow(b,2));
  cout<< "y ="<<y;
  file1.close();
```

```
 getch();
ofstream file2("MyRez.txt");
file2<< y;
file2.close();
     return 0;
}
```

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
2	k = N mod 4 + 1; dani.txt: «0 0 3k 0 0 4k»	k = 4: «0 0 12 0 0 16»
3	MyData.txt: x_1 = N mod 5 + 1,5; x_2 = 0,5 + 0,25·(N mod 4)	«3.5 1.25»

б) Кожне завдання — окремий файл Прізвище_ПР73_зN.cpp (наприклад, Ivanenko_PR73_z2.cpp), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно. Файли dani.txt і MyData.txt додайте до здачі.

в) Компілюйте й запускайте програми в Code::Blocks, Dev-C++, Visual Studio чи іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот вікна виконання кожної програми з введеними даними.

г) Підключайте <iostream> (за потреби <cmath>, <iomanip>) і using namespace std; заголовки <iostream.h> і <conio.h> із прикладів застаріли — замість getch() використовуйте cin.get() або system("pause"). Якщо кирилиця в консолі спотворюється, додайте SetConsoleCP(1251); SetConsoleOutputCP(1251); (<windows.h>). Тексти виводу — точно як у прикладах. Дані читайте з файлів засобами ifstream (<fstream>) або перенаправленням; результат виводьте на екран із чотирма знаками після коми (fixed, setprecision(4)).

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Хід роботи

1. Створіть потрібні файли в текстовому редакторі «Блокнот» для прикладів 5-7.

Перевірте роботу програм наведених в даних прикладах.

2.Створіть текстовий файл dani.txt , який містить значення координат вершин трикутника в наступному порядку x1, y1, x2, y2,x3, y3, які розділені прогалинами . Створіть програму, яка зчитавши вхідні дані зі створеного файлу, обчислить периметр і площу даного трикутника і надрукує результат на екрані.

Приклад (N = 7): dani.txt: «0 0 12 0 0 16» → «P = 48.0000», «S = 96.0000».

3. Дано файл MyData.txt, який містить два дійсних числа. Скласти програму, яка порахує величину w1 для першого числа і w2 для другого числа з даного файлу. Значення w1 і w2 обчислюються за формулами:

Приклад (N = 7): MyData.txt: «3.5 1.25» → «w1 = 7.0295», «w2 = 3.9204» (w1 — для першого числа, w2 — для другого). Якщо файл не відкрився — «Файл не знайдено».

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Завдання 1	2 бали	2
Завдання 2	4 бали	4
Завдання 3	5 балів	5
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, не перевірено відкриття файлу, неправильна формула, неправильний формат результату) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити дані у файлі чи формулу). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Як відкрити текстовий файл для читання засобами C++ (ifstream)?
2. Як перевірити, що файл успішно відкрито?
3. Як записати результат роботи програми у файл?
4. Чим відрізняється читання з файлу оператором >> від читання з клавіатури?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.